Sachai Check
Audio Deepfake Module

A beginner's guide to what we built, how it works, what results we got, and what still needs fixing.

What	Where it lives
Project folder	D:\Sachai Check\deepfake audio\sachai-audio\sachai-audio
The trained run	runs/xlsr300m-sls-v1
Every number in this guide comes from	runs/xlsr300m-sls-v1/report/ and export/ (real A4000 run)
Prepared	10 November 2026

How to read this guide

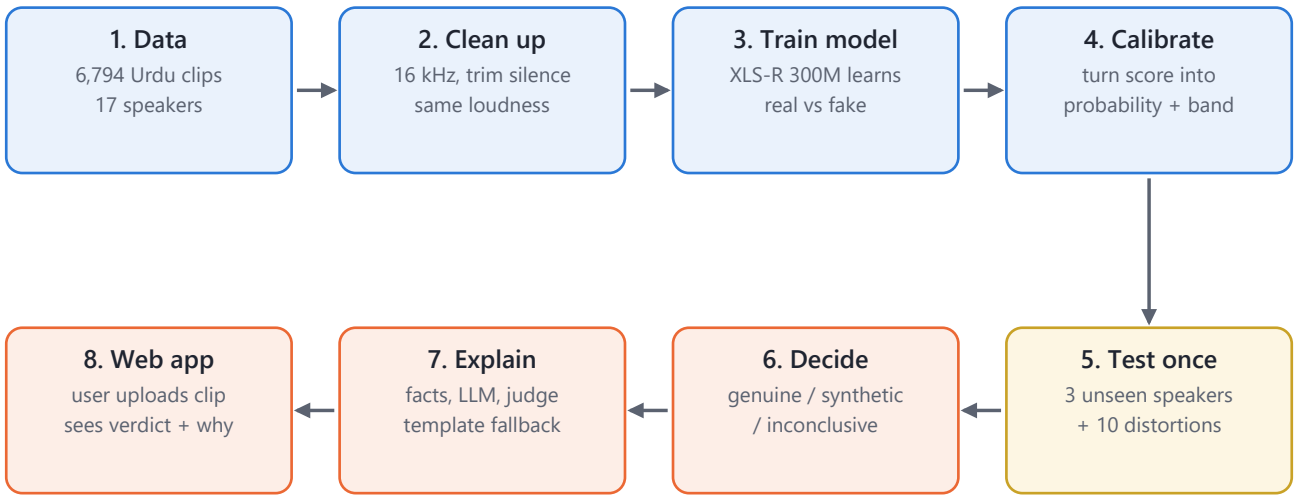
Sections 1 to 2 give you the big picture and the vocabulary. Sections 3 to 9 walk through the pipeline in the order the data flows. Section 10 shows the results. Section 11 lists the problems honestly. Section 12 says what to do next. Section 13 is a one-page cheat sheet.

Blue boxes explain. Orange boxes warn. Green boxes mean something is working well.

01 The big picture

Somebody sends you a WhatsApp voice note in Urdu. It sounds like a politician, or your uncle, or a news anchor. Is it really them, or did a computer generate it? That is the question this module answers.

We built a system that listens to a clip and says one of three things: **genuine**, **synthetic** (computer-made), or **inconclusive** (not sure enough to say). It also explains why, in plain language, using only things it actually measured.



Top row: research side (done once, offline). Bottom row: what happens for every clip in the product, plus the one-time honest test (yellow) that tells us how good it really is.

Two goals at once

- **A product**: a web page where a user uploads a clip and gets a verdict with reasons.
- **A research paper**: so every number we report must be honest and reproducible. We never tune the system on the final test to make numbers look better.

What we deliberately kept simple

One model, one training run, no comparisons against other models, no repeated runs with different seeds. This was your decision, and it keeps the project focused and finishable.

02 Words you need first

Read this table once. Every later section uses these words.

Word	What it means in plain words
Bona fide / genuine	A real human recording. Label 0.
Spoof / synthetic	Audio made by a computer to sound like a person. Label 1. This is the thing we hunt for.
TTS	Text-to-speech. You type text, the computer speaks it in someone's voice.
Tacotron, VITS	Two TTS systems. Our fake clips were made with them. Tacotron is older; VITS is newer and more natural.

Word	What it means in plain words
Sample rate	How many numbers per second describe the sound. We use 16,000 (16 kHz) for everything.
Model	A big mathematical function with millions of adjustable knobs (parameters). Ours has about 300 million.
Training	Showing the model thousands of labelled clips so it adjusts its knobs to tell real from fake.
Epoch	One full pass over all training clips.
Loss	A number that says how wrong the model is. Lower is better.
Train / dev / test	Three separate groups of speakers. Train teaches. Dev helps choose settings. Test is the final exam, opened once.
Score	The model's output between 0 and 1. Near 0 means 'looks real', near 1 means 'looks fake'.
Threshold	The cut-off on the score where we switch our answer.
EER	Equal Error Rate. Pick the threshold where 'real called fake' equals 'fake called real'. That shared error rate is the EER. Lower is better; 0% is perfect.
AUC	Chance that a random fake clip gets a higher score than a random real clip. 1.0 is perfect, 0.5 is coin-flipping.
FAR (here)	Share of real clips wrongly called fake. A false alarm. Embarrassing for the product.
FRR (here)	Share of fake clips wrongly let through as real. A miss. Dangerous for the product.
Calibration	Making the score mean what it says: of all clips scored 0.8, about 80% should truly be fake.
ECE	Expected Calibration Error. How far scores are from honest probabilities. 0 is perfect.
Coverage	Share of clips where the system gave a firm answer (not inconclusive).
OOD	Out of distribution. The clip looks unlike anything seen in training, so we warn the user.

03 The data

We use the public Hugging Face dataset **CSALT/deepfake_detection_dataset_urdu**. It has Urdu speech from 17 speakers.

Type	Clips	Speakers	Notes
Genuine (Bonafide)	3,398	17	Part 1 and Part 2 per speaker. Part 2 uses the same sentences as the fakes.
VITS fakes (Spoofed_TTS)	1,698	17	Newer TTS, sounds more natural
Tacotron fakes	1,698 (1,679 kept)	17	19 clips shorter than 0.5 s were dropped
Total	6,794	17	The original paper reports 37,281 clips; the public release is smaller

Why we split by speaker, not by clip

If the same person appears in both training and test, the model can cheat by recognising the *voice* instead of learning what *fakeness* sounds like. So we split people, not clips:

Group	Speakers	Clips	Job
Train	S02 S03 S04 S05 S07 S08 S11 S12 S13 S14 S15 S16	4,787	Teaches the model
Dev	S06 S10	798	Picks the best epoch, sets calibration and thresholds
Test	S01 S09 S17	1,190	Final exam. Scored once with everything frozen

This split is written once in `data/splits.json` and never regenerated.

The shortcut audit (checking the data is not lying to us)

Datasets often contain accidental clues. Before training, we checked whether simple things like clip length, silence or loudness could reveal the label without listening to the voice at all.

Group	Original sample rate	Silence at start	Silence at end	Loudness
Genuine	16 kHz and 20 kHz	0.06 s	0.02 s	-27 dBFS (quieter)
Tacotron	20 kHz	0.27 s	0.25 s	-18 dBFS
VITS	16 kHz	0.08 s	0.64 s	-15 dBFS

What the audit found

On the raw files, a simple classifier using only this metadata separates real from fake with **AUC 1.0**. That is a perfect cheat: silence and loudness give the answer away.

After our clean-up (resample to 16 kHz, trim silence, equalise loudness), the same check drops to **AUC 0.63**. Our red line is 0.75, so we are below it. The model now has to listen to the speech itself.

04 Cleaning the audio (preprocessing)

Every clip, during training and in the live product, goes through exactly the same steps in `audio_io.py`. If the product prepared audio differently from training, the model would see something unfamiliar and fail.

- **Decode** any format (wav, mp3, m4a, ogg, webm, flac) using ffmpeg.
- **Mono and 16 kHz**: one channel, 16,000 samples per second.
- **Trim silence** at the start and end (anything 40 dB quieter than the loudest part). This removes the silence shortcut.
- **Normalise loudness** so real and fake clips are equally loud. This removes the loudness shortcut.
- **Cut into windows**: 4-second windows with 2-second steps for the verdict, 2-second windows for the timeline.

This recipe has a version tag, `v1-16k-mono-trim40`. If it ever changes, the model must be retrained.

Augmentation: making training harder on purpose

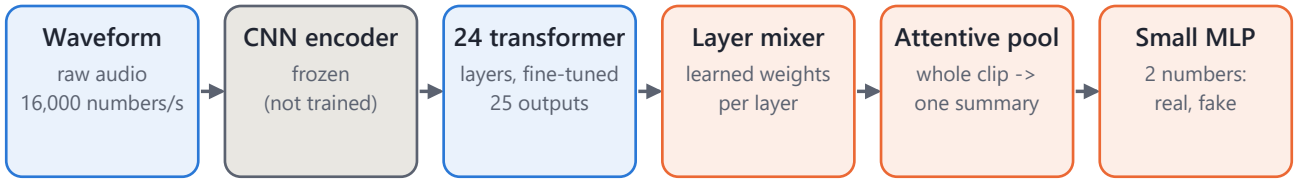
During training, 75% of clips get one or two random distortions: background noise (5 to 30 dB), room echo, telephone-style band limiting, mu-law (old phone line), EQ changes, or clicks. Each training clip is a random 2, 3 or 4 second crop.

Why 'label-blind' matters

The same distortions are applied to real and fake clips with the same probability. If only fakes got noise, the model would learn 'noise means fake', which is a shortcut, not knowledge.

05 The model

We did not build a model from zero. We started from **XLS-R 300M** (`facebook/wav2vec2-xls-r-300m`), a model Meta trained on about 436,000 hours of speech in 128 languages. It already 'understands' what human speech sounds like. We teach it the last step: spotting fakes.



Blue/grey = the pretrained XLS-R backbone (from Meta). Orange = the small parts we added on top.

The parts, one by one

Part	What it does	Trained by us?
CNN encoder	Turns raw sound into short frames (about 50 per second)	No, frozen
24 transformer layers	Build richer and richer understanding of each frame. Gives 25 outputs (input + 24 layers)	Yes, gently (learning rate 1e-5)
Layer mixer (SLS idea)	Learns how much to trust each of the 25 outputs, then blends them	Yes
Attentive statistics pooling	Decides which moments matter most, then summarises the whole clip into one vector (average + spread)	Yes
2-layer MLP head	Turns the summary into two numbers: 'real' and 'fake'	Yes (learning rate 5e-4)

The difference between the two output numbers is the **spoof log-odds**. Positive means 'fake is more likely'. Later we turn it into a score between 0 and 1.

In plain words

Imagine an expert listener who knows Urdu speech very well (XLS-R). We sit next to them for a few hours, play real and fake clips, and say which is which. They do not forget what speech is; they just learn the tell-tale signs of Tacotron and VITS.

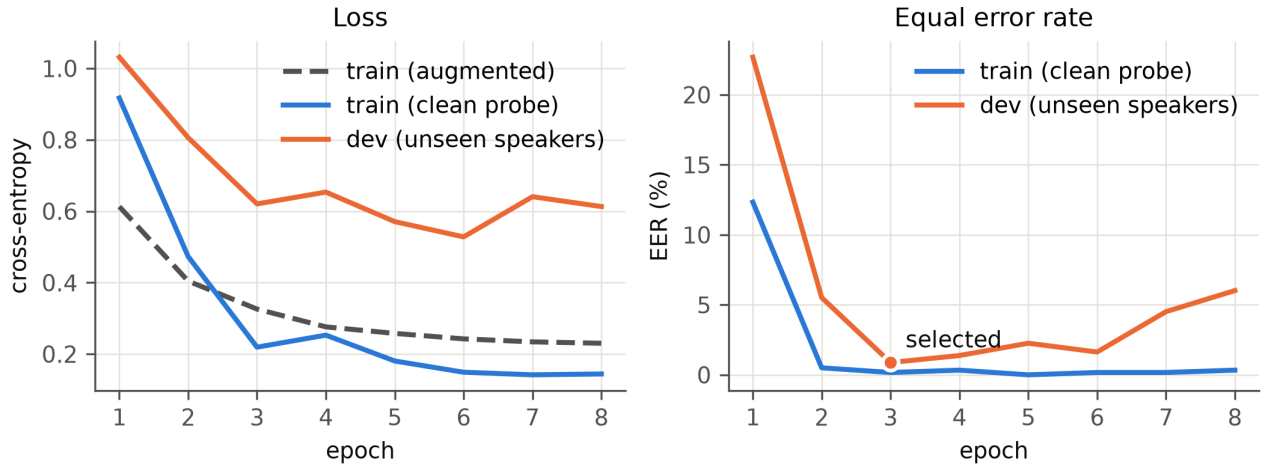
06 Training

Setting	Value	Why
Batch	16 clips x 2 accumulation = 32	Fits the 16 GB A4000
Precision	bf16 + gradient checkpointing	Saves memory, keeps speed
Max epochs / patience	20 / 5	Stop if dev does not improve for 5 epochs
Label smoothing	0.05	Stops the model being over-confident
Selection rule	lowest dev EER	Best epoch is chosen on dev speakers only

Surviving power cuts

A full checkpoint (weights, optimiser, position in the data, random state) is saved every 8 minutes and at the end of each epoch. Files are written safely so a half-written file is detected and the previous one is used. After a crash you just run `run.py train` again and it continues from the exact batch.

What actually happened



Left: loss. Right: EER. Blue = training speakers (clean), orange = dev speakers the model never trained on.

Epoch	Train-probe EER	Dev EER	Dev AUC	
1	12.3%	22.7%	0.888	
2	0.50%	5.5%	0.987	
3	0.17%	0.88%	0.9995	best, kept
4	0.33%	1.4%	0.9993	
5	0.00%	2.3%	0.998	
6	0.17%	1.6%	0.999	
7	0.17%	4.5%	0.990	
8	0.33%	6.0%	0.981	stopped (5 epochs, no gain)

How to read this

The model learned fast. By epoch 3 it was nearly perfect on both groups. After that, it kept getting better on training speakers but slowly *worse* on new speakers (dev EER went from 0.88% to 6.0%). That is the start of overfitting: memorising the training voices.

Early stopping handled it: we kept epoch 3. At that point the train/dev gap is 0.7 points, well under our 5-point red line. No underfitting either (train EER 0.17%, far below 5%).

07 From score to verdict

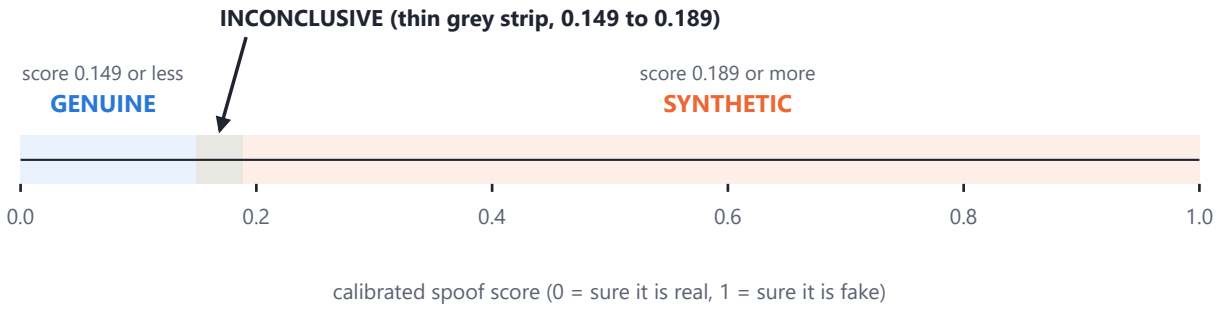
The raw model output is not a trustworthy probability. Two steps fix that, both fitted on dev speakers only.

Step 1. Temperature scaling (calibration)

We divide the raw log-odds by a number T, then squash it into 0 to 1. T was fitted so dev scores are as honest as possible. Our T is **2.23**. It is inside the allowed range (0.2 to 10), so it did not get stuck.

Step 2. The three-way decision

Instead of one cut-off, we use two. Between them, the system admits it is unsure.



- **EER threshold on dev**: 0.169.
- **Genuine line**: 0.149. **Synthetic line**: 0.189. These target 99% precision on each side, and the band is at least plus or minus 0.02 around 0.169.
- The band was first set to plus or minus 0.10. On dev, that produced no genuine verdicts at all, so it was narrowed to 0.02 before the test set was opened.

Why scores for real clips sit near 0.15 and not near 0

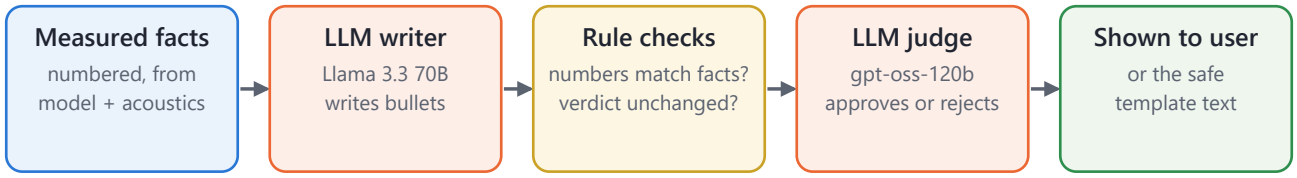
Temperature 2.23 and label smoothing squeeze scores toward the middle. A typical real clip scores about 0.14, a typical Tacotron fake about 0.85. So '0.15' does not mean '15% fake' in a strong sense. That is also why calibration (ECE about 0.21) is weak. See section 11.

Step 3. The out-of-distribution check

The model's internal summary of a clip is compared with summaries of training clips. If the new clip is far from everything seen before (for example music, a different language, heavy distortion), the result carries an OOD warning so the user does not over-trust it.

08 Explaining the verdict

A verdict alone is not useful. A user wants to know *why*. But an LLM can easily make things up. So we built the explanation in a way that keeps it honest.



Any failure (no key, provider error, rule broken, judge says no) falls back to the template. The verdict is never touched.

- **Facts first**: the system writes numbered facts it actually measured, for example the score, how many 2-second segments were flagged, and acoustic measurements.
- **LLM writer** (Groq, Llama 3.3 70B) turns facts into short bullets and an Urdu summary. It may cite only fact numbers.
- **Rule checks** in code: every number in a bullet must match a fact, and the bullet must agree with the verdict.
- **LLM judge** (a second model, gpt-oss-120b) reads the bullets against the facts and approves or rejects.
- **Template fallback**: if anything fails, fixed sentences built straight from the facts are shown instead.

> **The golden rule**
> The explanation can never change the verdict. The detector decides; the LLM only describes.

The 7 acoustic descriptors

These are classic measurements that give context. They are not what the model uses to decide. Cliff's delta shows how different a fake group is from genuine: 0 means no difference, plus or minus 1 means complete separation, and about 0.47 or more counts as a large effect.

Descriptor	Genuine	Tacotron	Delta	VITS	Delta
Pitch variation (semitones)	1.55	1.27	-0.40	1.42	-0.22
Frame-to-frame pitch movement	0.125	0.100	-0.49	0.200	+0.08
Share of voiced frames	0.885	0.930	+0.41	0.844	-0.06
Share of clip in pauses	0.018	0.080	+0.47	0.010	-0.15
Background level in pauses (dB)	40.1	42.5	+0.40	37.3	-0.51
High vs low frequency energy (dB)	-25.4	-37.2	-0.44	-29.1	-0.11
Spectral flatness	0.00083	0.00023	-0.52	0.00158	+0.08

Reading it: Tacotron is clearly different on almost everything (flatter pitch, more pauses, less high-frequency energy). VITS looks much more like real speech; only the background level in pauses stands out. That matches the results: VITS is the harder attack.

09 The product

A FastAPI server (`api/main.py`) loads the exported model once and serves the web page (`frontend/index.html`). The Audio Deepfake tab is live; the news tab is still demo data.

Item	Detail
Upload	wav, mp3, m4a, ogg/opus, webm, flac. 3 to 60 seconds. Up to 20 MB.
You get back	verdict, Urdu and English label, score, a 2-second timeline with flagged parts, audio quality, OOD warning, descriptors, explanation bullets, caveats
Speed	112 ms for a 7.7 s clip on the A4000 (from the demo run)
Privacy	Upload goes to a private temp folder and is deleted right after. Nothing is stored, logged or used for training. Only numbered facts (numbers, not audio) go to the LLM provider.
Error codes	413 too big, 415 bad format, 422 too short/long, 429 busy, 503 no model, 504 timeout

The demo run, as it happened

Clip	Truth	Score	Verdict
Spoofed_Tacotron/Speaker_01/10.wav	synthetic	0.794	Likely synthetic (8 of 8 segments flagged)
Bonafide/Speaker_01/Part 1/10.wav	genuine	0.151	Inconclusive (just 0.002 above the genuine line)

Both demo explanations came from the template, not the LLM, even though an LLM key is set in `.env`. See section 11.

10 Results

The test set (speakers S01, S09, S17) was scored once, with the model, temperature and thresholds frozen.

Metric	Dev (2 speakers)	Test (3 unseen speakers)	What it says
EER	0.88%	8.57% (95% CI 7.1 to 9.9)	Ranking is good, but much worse on new voices
AUC	0.9995	0.981 (95% CI 0.976 to 0.985)	Fakes still almost always score higher than reals
Accuracy (one cut-off)	99.1%	83.8%	Big drop, see the S17 story below
Real called fake (FAR)	1.0%	32.2% (193 of 600)	The main weakness
Fake let through (FRR)	0.75%	0.0% (0 of 590)	No fake was missed
Coverage	78.9%	89.1%	Share of firm answers
ECE	0.208	0.219	Scores are not honest probabilities yet

Per attack

Attack	Fake clips	EER	AUC	Caught as synthetic
Tacotron	290	0.26%	0.9999	100%
VITS	300	11.6%	0.963	99.3% (0.7% inconclusive)

Per speaker: the S17 story

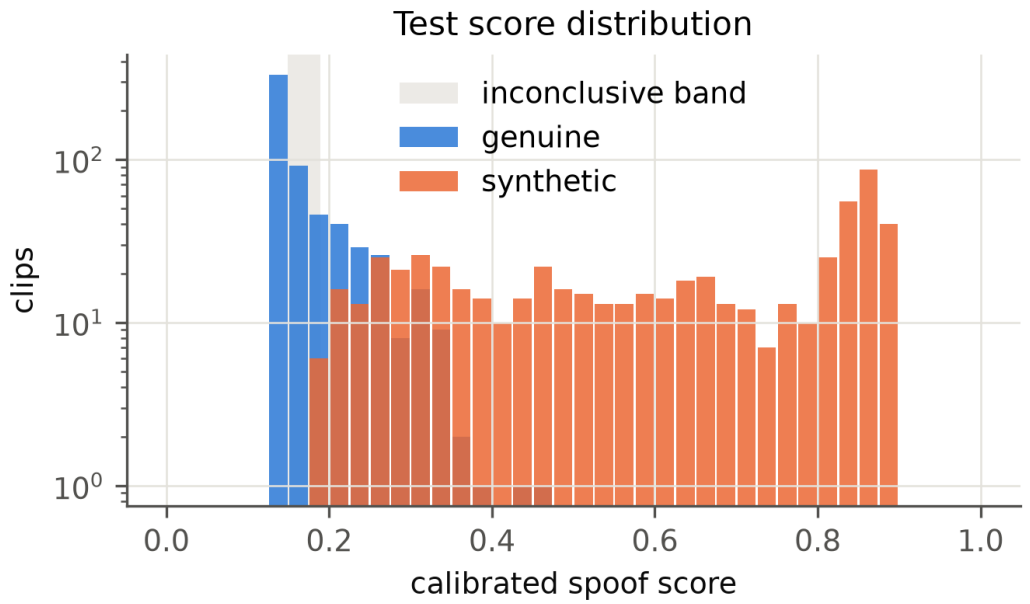
Speaker	Clips	EER	AUC	Real clips: genuine / unsure / called fake
S01	393	0.0%	1.000	161 / 39 / 0
S09	397	0.25%	0.9999	152 / 47 / 1
S17	400	2.0%	0.9985	**2 / 42 / 156**

What is going on with S17

The model still *ranks* S17 correctly: within S17, fakes score higher than reals (AUC 0.9985). But S17's real voice sits higher on the score scale. Their median real score is 0.223; for S01 and S09 it is about 0.145.

Our synthetic line is 0.189, chosen on dev speakers. For S17, almost every real clip crosses that line. Result: 156 of 200 real S17 clips are called fake. Nearly all 193 test false alarms come from this one speaker.

Lesson: one fixed threshold learned from only 2 dev speakers does not transfer to every new voice.



Test scores. Blue = real, orange = fake, grey strip = inconclusive band. The blue tail to the right of the band is mostly S17.

Robustness: what happens when audio is messy

We took the test clips and distorted them in 10 ways, then scored them with the same frozen settings.

Condition	EER	AUC	Real called fake	Fake missed
Clean	8.6%	0.981	32%	0%
Noise 20 dB (light)	12.2%	0.963	44%	0.2%
Noise 10 dB	18.7%	0.911	74%	0%
Noise 5 dB (heavy)	21.5%	0.870	92%	0%
Echo, small room (0.3 s)	19.6%	0.910	71%	0%
Echo, big room (0.8 s)	22.3%	0.892	98%	0%
Telephone 8 kHz	10.8%	0.969	32%	0%

Condition	EER	AUC	Real called fake	Fake missed
MP3 64 kbps	9.6%	0.979	32%	0%
MP3 32 kbps	8.2%	0.983	34%	0%
Opus 16 kbps (voice note)	6.8%	0.988	36%	0%

Reading the robustness table

Good news: compression (MP3, Opus WhatsApp-style voice notes) and phone quality barely hurt. That is the most common real-world case.

Bad news: noise and echo push real clips over the line. With heavy noise or a big room, almost every real clip is called fake. The model has partly learned 'degraded audio looks fake'.

11 Issues to resolve

Ordered from most to least important. Nothing here is hidden in the numbers; it should all be reported.

#	Issue	Why it matters	What to do
1	False alarms on some real voices (S17)	Real people get accused of being fake. Worst failure for a product.	Threshold from 2 dev speakers does not transfer. Candidate fix (v1.1 policy, below). Report the v1 test result as is.
2	Noise and echo make real clips look fake	Real voice notes recorded in a noisy street or echoey room may be flagged.	Short term: show a caveat when the quality check sees noise or echo. Long term: a new run with stronger augmentation (needs your approval).
3	Weak calibration (ECE about 0.21)	Scores like 0.15 or 0.8 do not mean what they appear to.	Temperature scaling can only stretch scores, not shift them. Platt scaling (v1.1) brings dev ECE to about 0.05.
4	VITS is much harder than Tacotron	Test EER 11.6% vs 0.26%. Newer TTS is harder; modern cloning will be harder still.	Report it. Keep the product caveat that modern voice cloning was never in training.
5	Layer weights did not learn	All 25 weights stayed between 0.0395 and 0.0408 (uniform is 0.040). The layer-weight figure shows no real preference.	Do not claim layer insights in the paper. Say the model used a near-even blend of layers.
6	Many real clips are inconclusive	On dev, 35% of real clips get 'unsure'. The demo's real clip missed by 0.002.	Part of the v1.1 policy choice (balanced or lenient trades unsure for more errors).
7	Only 2 dev speakers	Every fitted value (T, thresholds) rests on two voices.	Cannot change without breaking the frozen split. Disclose it; use the 'unseen speaker' check below.
8	Explanations fell back to template in the demo	Users see plain template text, not the LLM write-up.	Check the `llm_error` and `judge` fields of an API response to see if it is a provider error or a judge rejection.
9	Model not on the Hub yet	No push log in logs/. If this PC dies, the model is gone.	Run `run.py push` (after you approve publishing).

#	Issue	Why it matters	What to do
10	Housekeeping	CLAUDE.md section 15 still says the real run is not done. REPORT_CHANGES.md has \todo{} numbers to fill.	Update both from the files in report/.

The v1.1 policy candidates (already measured, dev only)

A comparison was already run with `run.py options`. It used dev speakers only and never read test scores. The useful rows are 'unseen speaker': fit on one dev speaker, measure on the other. That mimics a new voice.

Option (unseen speaker)	Real: genuine	Real: unsure	Real: called fake	Fake: called genuine	VITS called genuine	ECE
v1 (current)	51.4%	48.6%	0.0%	3.0%	6.0%	0.242
v1.1 strict	52.6%	47.1%	0.3%	3.0%	6.0%	0.048
v1.1 balanced	70.7%	28.1%	1.3%	5.8%	11.6%	0.048
v1.1 lenient	87.2%	9.8%	3.0%	7.8%	15.6%	0.048

Every v1.1 option fixes calibration. The trade-off is simple: fewer 'unsure' answers for real people means more fakes slipping through as genuine, especially VITS.

An honesty point you must keep

The v1.1 idea came *after* we saw the S17 problem on test. Even though v1.1 is fitted on dev only, its test number would no longer be a clean, blind result.

So for the paper: report v1 on test as the main result. If the product moves to v1.1, describe it as a later product change, and measure it on **new data** (new speakers, never seen) rather than on the old test set.

12 What to do next

Step	What	Who decides
1	Push the model to the Hugging Face Hub so it stays alive (`run.py push`).	You approve first
2	Pick a product policy: keep v1, or switch to v1.1 strict / balanced / lenient.	You
3	Find out why the LLM explanation fell back to template, and fix the key or prompt.	Can be done now
4	Add a product caveat when audio is noisy or echoey (quality check already measures it).	Can be done now
5	Record a small fresh set: new Urdu speakers, phone mics, WhatsApp notes, plus clips from a modern voice-cloning tool. This is the only honest way to judge v1.1 and real-world accuracy.	Team
6	Fill REPORT_CHANGES.md numbers from report/ files and update CLAUDE.md status.	Can be done now
7	Optional, later: a new run (new run name) with stronger noise and echo augmentation, chosen on dev only.	You (it is a retrain)

Things we must never do

Change thresholds, temperature or anything else by looking at test or robustness results.

Retrain several times and keep whichever run tests best.

Regenerate `data/splits.json` or mix speakers between groups.

Hide S17, the noise weakness, or the 'no modern cloning in training' limit.

13 Cheat sheet

Commands (from the project folder, in Git Bash)

```
.venv/Scripts/python run.py check        # GPU, ffmpeg, disk
.venv/Scripts/python run.py data         # download + cache + split
.venv/Scripts/python run.py audit        # shortcut check
.venv/Scripts/python run.py train        # resumes after a crash
.venv/Scripts/python run.py fit          # learning curves + diagnosis
.venv/Scripts/python run.py finalize     # calibrate + export
.venv/Scripts/python run.py options      # compare v1 / v1.1 on dev only
.venv/Scripts/python run.py test         # final exam, once
.venv/Scripts/python run.py robustness   # 10 distortions
.venv/Scripts/python run.py analysis     # descriptors + layer weights
.venv/Scripts/python run.py demo         # two clips + speed
.venv/Scripts/python run.py push         # upload to the Hub (ask first)
.venv/Scripts/python -m uvicorn api.main:app --host 127.0.0.1 --port 8000
```

Where things are

Path	Contains
data/splits.json	Who is train, dev, test. Never touch.
runs/xlsr300m-sls-v1/history.csv	One row per epoch
runs/.../export/	The product model: weights, policy.json, OOD stats, reference stats
runs/.../report/table_main.csv	Dev vs test headline numbers
runs/.../report/test_per_speaker.csv	Where S17 shows up
runs/.../report/table_robustness.csv	The 10 distortions
runs/.../report/policy_options_dev.csv	v1 vs v1.1 comparison (dev only)
runs/.../report/*.tex, fig_*.pdf	Ready for the paper
sachai_audio/	All code. See the code map in CLAUDE.md section 10

14 Final verdict

Where the project stands

The pipeline works end to end: data, training, calibration, testing, robustness, explanation and the live web app. The model is genuinely good at *ranking* real against fake on new voices (test AUC 0.981) and missed zero fakes on test.

It is not ready to be trusted blindly. It wrongly accuses one test speaker, it confuses noisy or echoey real audio with fakes, its scores are not well calibrated, and it has never seen modern voice cloning. All of this is fixable or at least reportable, and none of it requires breaking the rules of honest evaluation.

Verdict: a solid, honest research prototype. Push the model, choose the product policy, collect a small fresh test set, and report the weaknesses openly.